SE3002 — Software Quality Engineering

Assignment 2 — Final Testing Report

Student Course Registration System

Black-box testing · White-box testing · Defect management · GitHub

collaboration

Repository	https://github.com/Maaz976/Python-Registration-Testing
System under test	`registration.py` (Student Course Registration System)
Oracle	`REQUIREMENTS.md` / `Student_Registration_SRS.docx` (FR-01–FR-15)
Final regression	**80 passed, 0 failed · 100% statement and branch coverage**
Open PRs	PR #6 — black-box harness and structure PR #7 — defect fixes and regression evidence
Prepared by	Pir Ahmed Shah (`PirAhmedShah`) · Repository owner: `Maaz976`

1. Executive summary

This report documents requirements-based black-box design, structural white-box analysis, defect reporting, implementation fixes, and regression evidence for the supplied Student Course Registration System. Expected results were derived from the SRS rather than from the implementation.

Area	Result
A1 — Equivalence Class Partitioning	20 cases covering valid/invalid credits, dues, prerequisites, availability, duplicate registration, and validation priority.
A2 — Boundary Value Analysis	25 cases covering current credits 0–18, course credits 1–4, combined 18-credit maximum, capacity/enrollment, and dues around zero.
A3 — Decision Table	6 rules and 10 executable cases respecting the FR-09 priority: dues → duplicate → prerequisites → credit limit → availability.
A4 — Black-box pytest	64 executions. Initial run: **49 passed, 15 failed**; failures preserved as defect evidence.
A5 — Optimisation	0 exact duplicates after applying an operation + complete pre-state + expected-result criterion; all 55 design cases retained.
B1 — CFG and cyclomatic complexity	Short-circuit-expanded CFG; V(G) = 27 − 19 + 2 = 10; 10 independent basis paths; all 10 executable after fixes.

B2 — Statement/branch coverage	Initial 94.74% statement / 88.46% branch; after white-box additions 96.49% / 92.31%; after fixes **100% / 100%**.
B3 — Condition coverage and MC/DC	9 atomic conditions analysed with T/F/NE truth tables; independent-effect pairs for the compound validation guard produce 3/3 masking MC/DC.
B4 — White-box pytest	11 structural cases plus 5 focused regression cases (`REG-01`–`REG-05`).
C — Collaboration and defects	5 GitHub issues (#1–#5), 2 open PRs (#6, #7), descriptive commits, and GitHub Actions evidence including legitimate pre-fix failures.
D — Final regression	**80 passed, 0 failed**; 54/54 statements and 26/26 branch arcs covered (100%).

2. Repository and method

2.1 Required structure

```
SQE-Assignment-2/
├── registration.py
├── requirements.txt
├── README.md
├── REQUIREMENTS.md
├── pytest.ini
├── tests/
│   ├── test_blackbox.py
│   └── test_whitebox.py
├── docs/
│   ├── blackbox_test_design.xlsx
│   ├── decision_table.xlsx
│   ├── cfg.png
│   ├── testing_report.pdf
│   ├── White-Box-Testing.docx
│   ├── testing_optimization_and_coverage.md
│   └── evidence/
└── .github/workflows/tests.yml
```

The test suite was standardised on the required plural `tests/` path and the CFG artifact on `docs/cfg.png`. Historical evidence keeps the paths that were current when each run was captured.

2.2 Workflow used

1. Read the SRS and derive expected outcomes before inspecting implementation branches.
2. Design ECP, BVA, and decision-table cases in workbooks and implement them in `tests/test_blackbox.py`.
3. Execute the black-box suite, preserve all failures, and open one GitHub Issue per distinct

defect.

4. Inspect source code, draw the CFG, calculate V(G), and design condition/MC-DC analysis.
5. Add structural tests in `tests/test_whitebox.py` and measure initial/final coverage.
6. Fix confirmed defects on `defect-fixes`, one commit per defect, and rerun the complete suite.
7. Record final coverage and open pull requests without merging, preserving review and Actions evidence.

3. Part A — Black-box testing

3.1 A1 — Equivalence Class Partitioning

The ECP workbook records requirement ID, class type, representative input, expected output, expected state change, and test ID. Twenty cases (`ECP-01–ECP-20`) cover the required dimensions.

Dimension	Valid classes	Invalid / blocked classes	Representative IDs
Current / course credits	Current 0–18; course 1–4; combined ≤ 18	Current < 0 or > 18; course < 1 or > 4	ECP-01–ECP-06
Outstanding dues	Rs. 0; positive numeric input	Negative dues; any balance > 0 blocks registration	ECP-07–ECP-09, ECP-20
Prerequisites	Empty required set; all required completed	Some or all required courses missing	ECP-10–ECP-13
Availability	`enrolled < capacity`	`enrolled == capacity` → waitlist; invalid capacity/enrollment raises	ECP-14–ECP-18
Duplicate registration	Course not already registered	Course already in `registered_courses`	ECP-19

3.2 A2 — Boundary Value Analysis

Twenty-five boundary cases (`BVA-01–BVA-25`) include at-boundary, just-inside, and just-outside values. Key cases:

Boundary	Values exercised	Expected outcomes	Test IDs
Current credits 0–18	−1, 0, 1, 17, 18, 19	ValueError; True; True; True; False; ValueError	BVA-01–BVA-06
Course credits 1–	0, 1, 2, 3, 4, 5	ValueError; True for 1–4; ValueError at 5	BVA-07–

4			BVA-12
Combined maximum 18	17, 18, 19	True; True; False	BVA-13–BVA-15
Capacity / enrollment	capacity 0/1/30; enrolled 0/1/29/30/31/−1	REGISTERED at open seat; WAITLISTED at exact capacity; ValueError outside valid region	BVA-16–BVA-22
Dues near zero	−1, 0, 1	ValueError; REGISTERED; DUES_BLOCK	BVA-23–BVA-25

3.3 A3 — Decision Table

The decision table uses FR-09 priority. A dash means the condition is not evaluated because an earlier rule determines the outcome.

Condition / action	R1	R2	R3	R4	R5
Outstanding dues > Rs. 0?	T	F	F	F	F
Already registered?	—	T	F	F	F
All prerequisites satisfied?	—	—	F	T	T
Credit total ≤ 18?	—	—	—	F	T
Seat available?	—	—	—	—	F
Expected outcome	**DUES_BLOCK**	**ALREADY_REGISTERED**	**PREREQUISITES_MISSING**	**CREDIT_LIMIT**	**WAITLIST**

Ten executable cases (`DT-01–DT-10`) cover every outcome and important priority combinations, including dues overriding duplicate registration and credit-limit failure overriding full capacity.

3.4 A4 — Black-box pytest execution

All 64 black-box executions are implemented in `tests/test_blackbox.py` with `assert` and `pytest.raises()`. State snapshots verify FR-10 and FR-12: unsuccessful decisions and failed registrations must not mutate either object.

Stage	Collected	Passed	Failed	Interpretation
Initial black-box baseline (unchanged implementation)	64	49	**15**	Legitimate failures against 5 confirmed defects
Black-box + white-box (still unchanged)	75	58	**17**	Two additional structural test IDs expose the same two defect classes
After defect fixes + regression cases	80	**80**	**0**	Complete pass

Evidence: `docs/evidence/blackbox-initial.txt`, `docs/evidence/whitebox-combined-coverage.txt`, and `docs/evidence/final-regression-coverage.txt`.

3.5 A5 — Test-suite optimisation

Duplicate criterion: two cases are exact duplicates only when they use the same operation, the same complete pre-state and concrete inputs, and require the same result/exception and state-change assertions. Sharing a requirement or expected status is not enough.

No exact duplicates were found across ECP, BVA, and decision-table suites. Similar-looking cases differ in credits, dues, capacity, enrollment, prerequisites, or priority conditions. Exact duplicate count = **0**; cases removed = **0**; all 55 design cases retained with their IDs and technique traceability.

4. Part B — White-box testing

4.1 B1 — CFG, cyclomatic complexity, and basis paths

The CFG expands Python short-circuit and chained comparisons into atomic predicate nodes. `registration_decision()` has 9 atomic decision nodes: D1 (negative dues), D2a/D2b/D2c (capacity guard), D3 (dues), D4 (duplicate), D5 (prerequisites), D6 (credit limit), and D7 (availability).

Method	Calculation	Result
Predicate nodes + 1	P = 9 atomic predicates	V(G) = 9 + 1 = **10**
Graph theory	N = 1 entry + 9 decisions + 8 terminals + 1 virtual exit = 19; E = 27	V(G) = 27 − 19 + 2 = **10**
Regions	9 bounded regions + 1 outer region	V(G) = **10**

Counting each source `if` once gives 7 decisions and V(G) = 8. The report explicitly uses the short-circuit-expanded convention so the CFG, condition table, and basis paths are consistent. The CFG image is `docs/cfg.png`.

Path	Decision sequence	Representative test	Expected outcome
1	D1(T) → raise	WB-CC-01	ValueError("Invalid dues")
2	D1(F) → D2a(T) → raise	WB-CC-03	ValueError("Invalid capacity or enrollment")
3	D1(F) → D2a(F) → D2b(T) → raise	WB-CC-04	ValueError("Invalid capacity or enrollment")
4	D1(F) → D2a(F) → D2b(F) → D2c(T) → raise	WB-CC-05	ValueError("Invalid capacity or enrollment")
5	D1–D2(F) → D3(T)	WB-CC-06 / WB-CC-11	DUES_BLOCK
6	D1–D3(F) → D4(T)	WB-CC-07	ALREADY_REGISTERED
7	D1–D4(F) → D5(T)	WB-CC-08	PREREQUISITES_MISSING

8	D1–D5(F) → D6(T)	WB-CC-09	CREDIT_LIMIT
9	D1–D6(F) → D7(T)	WB-CC-10 / REG-04	WAITLISTED
10	D1–D7(F)	WB-CC-02	REGISTERED

Before correction, path 9 was infeasible because D2b rejected `enrolled > capacity` and D7 tested the wrong condition. After the FR-08 fix, D7 tests `enrolled >= capacity` and all 10 basis paths are executable.

4.2 B2 — Statement and branch coverage

Measure	Black-box only (initial)	Black-box + white-box	After defect fixes
Tests collected	64	75	80
Passed / failed	49 / 15	58 / 17	**80 / 0**
Statement coverage	94.74%	96.49%	**100% (54/54)**
Branch coverage	88.46%	92.31%	**100% (26/26)**
Combined pytest-cov	92.77%	95.18%	**100%**
Missing locations	51, 59, 71	59, 71	**None**

The initial gaps were structural consequences of confirmed defects: line 51 (DUES_BLOCK) was reachable only through the incorrect `> 1000` threshold; lines 59 and 71 (WAITLISTED and waitlist append) were unreachable because the availability condition used `>` instead of `>=`. The fixes make those locations reachable without changing any expected value.

4.3 B3 — Condition coverage and MC/DC

The selected compound decision is the capacity validation guard *D2 = A or B or C*, where A = `enrolled < 0`, B = `enrolled > capacity`, and C = `capacity < 1`. Python short-circuits left-to-right. “NE” means not evaluated, not a third Boolean value.

Case	D1	A	B	C	D3	D4	D5	D6	D7	Outcome
WB-CC-01	T	NE	NE	NE	NE	NE	NE	NE	NE	Invalid dues
WB-CC-02	F	F	F	F	F	F	F	F	F	REGISTERED

WB-CC-03	F	T	NE	NE	NE	NE	NE	NE	NE	Invalid capacity
WB-CC-04	F	F	T	NE	NE	NE	NE	NE	NE	Invalid capacity
WB-CC-05	F	F	F	T	NE	NE	NE	NE	NE	Invalid capacity
WB-CC-06	F	F	F	F	T	NE	NE	NE	NE	DUES_BLOCK
WB-CC-07	F	F	F	F	F	T	NE	NE	NE	ALREADY_REGISTERED
WB-CC-08	F	F	F	F	F	F	T	NE	NE	PREREQUISITES_MISSING
WB-CC-09	F	F	F	F	F	F	F	T	NE	CREDIT_LIMIT
WB-CC-10	F	F	F	F	F	F	F	F	T	WAITLISTED
WB-CC-11	F	F	F	F	T	NE	NE	NE	NE	DUES_BLOCK

Atomic condition	True case	False case	Controlled change	Effect
A = enrolled < 0	WB-CC-03	WB-CC-02	enrolled −1 → 0; B and C logically false	Guard T → F
B = enrolled > capacity	WB-CC-04	WB-CC-02	enrolled 11 → 0; A and C remain false	Guard T → F
C = capacity < 1	WB-CC-05	WB-CC-02	capacity 0 → 10; A and B remain false	Guard T → F

Manual masking MC/DC for D2: 3/3 independent effects = 100%. Across the function's 9 atomic conditions, after the fixes each condition can evaluate both True and False (18/18 condition outcomes). `pytest-cov` does not measure condition coverage or MC/DC; these figures are manual design-level calculations.

4.4 B4 — White-box pytest implementation

`tests/test_whitebox.py` documents the targeted branch, condition, or path for every case. It contains `WB-CC-01–WB-CC-11` plus post-fix regression cases `REG-01–REG-05`. WB-CC-02 through WB-CC-05 reuse one baseline to demonstrate the three independent MC/DC effects without duplicating the baseline execution.

5. Part C — GitHub collaboration and defect management

5.1 Repository and branches

The repository contains the starter implementation, SRS, test suites, workbooks, CFG, evidence, final report, and workflow. Feature branches and descriptive commits include `blackbox-testing` (structure, pytest discovery, baseline evidence), `defect-fixes` (five focused fix commits plus regression coverage), and the existing `testing-documentation/testing-documentation-review` branches.

5.2 GitHub Issues

Issue	Defect	Requirement	Failed test IDs	Status
#1	Exactly 18 credits rejected	FR-01	BVA-04, BVA-14, DT-10	Fixed in PR #7; open pending merge
#2	Positive dues do not block	FR-04	ECP-08, BVA-25, DT-01, DT-02, STATE-03, WB-CC-11	Fixed in PR #7; open pending merge
#3	Partial prerequisites accepted	FR-03, FR-06	ECP-12	Fixed in PR #7; open pending merge
#4	Full course registers instead of waitlisting	FR-08, FR-12	ECP-15, BVA-17, BVA-19, DT-09, STATE-02, WB-CC-10	Fixed in PR #7; open pending merge
#5	Withdrawal does not decrement enrollment	FR-13	STATE-07	Fixed in PR #7; open pending merge

5.3 Pull requests and review

PR	Branch → base	Summary	Test outcome	Status
#6	`blackbox-testing` → `main`	Required tests/ structure, cfg.png, pytest.ini, real initial baseline evidence	49 passed / 15 expected failures; 92% combined	Open
#7	`defect-fixes` → `main`	Five defect fixes, regression cases, 100% coverage evidence, final report	80 passed / 0 failed; 100% coverage	Open

PR #7 links issues #1–#5 and states the dependency on PR #6. PRs are intentionally left open for review; no merge was performed. Review checklists cover correctness of SRS expectations, preservation of pre-fix evidence, focused commits, and regression results.

5.4 GitHub Actions

`.github/workflows/tests.yml` installs `requirements.txt`, runs the full pytest suite with statement and branch coverage, writes JUnit XML/XML/HTML reports, and uploads artifacts even when tests fail. Earlier runs failed legitimately on the five defects; the final `defect-fixes` run is expected to pass at 80/80 with 100% coverage. Actions history: repository Actions tab. The original failing run is preserved as run `43418723784`.

6. Part D — Defect status and final assessment

6.1 Defect fixes

Defect	Root cause	Correction	Regression verification
DEF-01	`current + new < 18`	`<= 18`	BVA-04, BVA-14, DT-10, REG-01
DEF-02	`dues > 1000`	`dues > 0`	ECP-08, BVA-25, DT-01, DT-02, STATE-03, WB-CC-11, REG-02
DEF-03	intersection instead of subset	`required.issubset(completed)`	ECP-12, ECP-13, REG-03
DEF-04	`enrolled > capacity` after rejecting invalid enrollment	`enrolled >= capacity` → WAITLISTED	ECP-15, BVA-17, BVA-19, DT-09, STATE-02, WB-CC-10, REG-04
DEF-05	missing `course.enrolled -= 1`	decrement on successful withdrawal	STATE-07, REG-05

6.2 Final coverage and quality outcome

Measure	Initial	Final	Change
Black-box tests	64 collected / 49 passed / 15 failed	64 / 64 passed	**15 defects resolved**
White-box tests	11 collected / 9 passed / 2 failed	16 / 16 passed	**+5 regression cases**
Statement coverage	94.74%	100% (54/54)	**+5.26 points**
Branch coverage	88.46%	100% (26/26)	**+11.54 points**
Combined pytest-cov	92.77%	100%	**+7.23 points**

> Issues #1–#5 remain open in GitHub because PR #7 was intentionally not merged. The

> implementation fixes and all regression evidence are present on the `defect-fixes` branch; issue closure is pending reviewer approval and merge.

6.3 Contribution summary

Member	GitHub	Contribution
Muhammad Maaz	`Maaz976`	Repository setup, black-box design workbook, initial evidence, CFG, white-box documentation, traceability matrix, defect issues.
Muhammad Muzammil Ayan	`MuhammadmuzammilAyan1`	Black-box pytest implementation, white-box pytest suite, coverage analysis, GitHub Actions workflow, optimisation and coverage review.
Pir Ahmed Shah	`PirAhmedShah`	Baseline reproduction, repository structure cleanup, five defect fixes, regression cases, final 100% coverage evidence, PRs #6/#7, final testing report.

6.4 Reproduction commands

```
python -m pip install -r requirements.txt
python -m pytest -v
python -m pytest tests/test_blackbox.py -v --cov=registration --cov-branch --cov-report=term-missing
python -m pytest tests/test_whitebox.py -v --cov=registration --cov-branch --cov-report=term-missing
python -m pytest tests/ --cov=registration --cov-branch --cov-report=term-missing
```

6.5 References

- Repository: https://github.com/Maaz976/Python-Registration-Testing
- Issues: https://github.com/Maaz976/Python-Registration-Testing/issues
- PR #6: black-box harness and repository structure
- PR #7: defect fixes and regression evidence
- Actions: GitHub Actions history
- Case design: `docs/blackbox_test_design.xlsx`, `docs/decision_table.xlsx`
- White-box analysis: `docs/White-Box-Testing.docx`, `docs/cfg.png`